RabbitMQ - Cheat Sheet

Contents

1. The core model
2. Exchange types
3. Queues
4. Message durability
5. Consumers and acknowledgement
6. Prefetch (QoS)
7. Reliability building blocks
8. Java client
9. RabbitMQ vs Kafka vs NATS
10. Clustering and HA
11. Best practices
12. Common gotchas
13. Quick reference

RabbitMQ is a message broker built on AMQP 0-9-1. Written in Erlang, first released in 2007, now maintained under VMware (Broadcom). A producer never talks to a queue directly. It publishes to an exchange, the exchange applies routing rules, and the message lands in one or more queues where consumers pick it up.

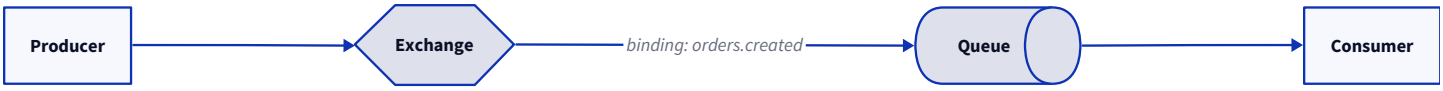
That indirection is the whole point. The broker is smart: it does the routing, buffering, retries, and dead-lettering. The consumer stays simple. This is the opposite of Kafka, where the broker is a dumb log and the consumer tracks its own position.

The core model

Five pieces move every message.

Piece	Role
Producer	Publishes a message with a routing key
Exchange	Receives the message, decides where it goes
Binding	A rule linking an exchange to a queue
Queue	Buffers messages until a consumer acks them
Consumer	Reads from a queue, acknowledges, or rejects

The flow reads left to right:



A producer publishes with an exchange name and a routing key. The exchange looks at its bindings and copies the message into every queue whose binding matches. If nothing matches, the message is dropped (or returned, if published as mandatory).

Exchange types

The exchange type decides how the routing key is matched against bindings.

Type	Routing rule	Use it for
`direct`	Routing key equals binding key exactly	Point-to-point, routing by a known key
`topic`	Routing key matched against a pattern with wildcards	Routing by category, most common choice
`fanout`	Ignores the routing key, copies to every bound queue	Broadcast, pub/sub
`headers`	Matches on message headers, not the routing key	Rare, when routing needs multiple attributes

There is also the default exchange: a nameless direct exchange that every queue is automatically bound to by its own name. Publish to routing key `orders.created` on the default exchange and the message goes straight to a queue named `orders.created`. Handy for quick point-to-point, but it hides the routing, so avoid it in real systems.

Topic wildcards

Routing keys are dot-separated words. Topic bindings use two wildcards:

- `*` matches exactly one word. `orders.*` matches `orders.created` but not `orders.us.created`.
- `#` matches zero or more words. `orders.#` matches both `orders.created` and `orders.us.created`, and even `orders`.

```
binding "orders.*.created"    matches    orders.us.created
binding "orders.#"            matches    orders.us.west.created
binding "*.created"           matches    users.created and orders.created
```

Queues

A queue holds messages in order until they are acknowledged. How it behaves depends on how you declare it.

Property	Effect
`durable`	Queue definition survives a broker restart
`exclusive`	Only the declaring connection can use it, deleted on disconnect
`auto-delete`	Deleted once the last consumer unsubscribes
`arguments`	TTL, max length, dead-letter target, queue type

Classic vs quorum vs streams

RabbitMQ has three queue types, and picking the wrong one is a common early mistake.

Type	What it is	When
Classic	Single-node queue, optional lazy mode	Simple, non-replicated workloads
Quorum	Raft-replicated, the modern default for HA	Anything that must survive a node loss
Stream	Append-only log, replayable, non-destructive reads	High throughput, replay, many readers

Quorum queues replaced classic mirrored queues, which are deprecated and removed in 4.0. If you need replication, reach for quorum. Streams are the Kafka-style option: consumers read by offset and the log stays put.

Message durability

Durability is a two-part contract, and people miss half of it. To survive a broker restart you need **both**:

1. A durable queue (the queue definition is persisted).
2. A persistent message (`delivery_mode = 2`).

A persistent message in a non-durable queue is lost on restart. A transient message in a durable queue is lost too. You need both, and even then durability means “written to disk”, not “replicated”. For replication, use a quorum queue.

Consumers and acknowledgement

A consumer either auto-acks or acks manually. This choice decides whether you can lose messages.

- **Auto-ack** (`autoAck = true`): the broker deletes the message the moment it is delivered. If the consumer crashes mid-processing, the message is gone. Fast, lossy.
- **Manual ack**: the consumer calls `basicAck` after it finishes. If it crashes first, the broker redelivers to another consumer. Safe, at-least-once.

Three ways to close out a message:

Call	Meaning
`basicAck`	Done, remove the message
`basicNack` / `basicReject` with `requeue=true`	Failed, put it back on the queue
`basicNack` / `basicReject` with `requeue=false`	Failed, dead-letter it (or drop)

Prefetch (QoS)

By default a consumer grabs as many unacked messages as the broker will hand out, so one slow consumer hoards the backlog while others sit idle. `basicQos(prefetchCount)` caps how many unacked messages a consumer can hold. Set it, always.

```
channel.basicQos(10);         // at most 10 unacked messages in flight per consumer
```

Start around 10 to 30 for typical work and tune from there. A prefetch of 1 gives the fairest distribution but the most round-trips.

Reliability building blocks

Publisher confirms

Publishing does not guarantee the broker stored the message. Turn on confirms and the broker acks each publish once it is safe.

```
channel.confirmSelect();
channel.basicPublish(exchange, routingKey,
        MessageProperties.PERSISTENT_TEXT_PLAIN, body);
channel.waitForConfirmsOrDie(5000);   // throws if the broker did not confirm
```

Pair confirms with the `mandatory` flag and a return listener to catch messages that no queue accepted.

Dead letter exchange (DLX)

A message is dead-lettered when it is rejected with `requeue=false`, expires on TTL, or the queue overflows its max length. The broker republishes it to the exchange named in the queue's `x-dead-letter-exchange` argument.

```
Map<String, Object> args = new HashMap<>();
args.put("x-dead-letter-exchange", "orders.dlx");
args.put("x-message-ttl", 60000);          // 60s, then dead-letter
args.put("x-max-length", 100000);          // overflow dead-letters too
channel.queueDeclare("orders.created", true, false, false, args);
```

DLX is how you stop a poison message from looping forever. Reject with `requeue=false`, let it land in the dead-letter queue, and inspect it out of band.

TTL

- Per-queue: `x-message-ttl` on the queue expires every message after the set time.
- Per-message: set `expiration` on the message properties.
- Queue TTL: `x-expires` deletes an unused queue after a period.

Java client

The official client is `com.rabbitmq:amqp-client`.

```
<dependency>
    <groupId>com.rabbitmq</groupId>
    <artifactId>amqp-client</artifactId>
    <version>5.x</version>
</dependency>
```

Publish

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost");

try (Connection conn = factory.newConnection();
     Channel channel = conn.createChannel()) {

       channel.exchangeDeclare("orders", BuiltinExchangeType.TOPIC, true);
       channel.queueDeclare("orders.created", true, false, false, null);
       channel.queueBind("orders.created", "orders", "orders.created");

       channel.basicPublish("orders", "orders.created",
               MessageProperties.PERSISTENT_TEXT_PLAIN,
               "{\"id\":42}".getBytes());
}
```

Consume with manual ack

```
channel.basicQos(10);      // prefetch

DeliverCallback onMessage = (consumerTag, delivery) -> {
     long tag = delivery.getEnvelope().getDeliveryTag();
     try {
         process(delivery.getBody());
         channel.basicAck(tag, false);
     } catch (Exception e) {
         // requeue=false sends it to the dead-letter exchange
         channel.basicNack(tag, false, false);
     }
};

channel.basicConsume("orders.created", false, onMessage, tag -> {});
```

Spring users usually skip this boilerplate and use Spring AMQP: `RabbitTemplate` for publishing and `@RabbitListener` for consuming, with confirms and retry wired through configuration.

RabbitMQ vs Kafka vs NATS

Aspect	RabbitMQ	Kafka	NATS JetStream
Model	Smart broker, routes per message	Dumb log, consumer tracks offset	Streams and consumers
Routing	Rich (exchanges, bindings, wildcards)	Topic and partition only	Subject hierarchy
Ordering	Per queue	Per partition	Per stream
Replay	No (once acked, it is gone)	Yes, by offset	Yes
Throughput	Medium	Very high	High
Protocol	AMQP 0-9-1 (also MQTT, STOMP)	Custom binary	Custom binary
Best for	Task queues, complex routing, RPC	Event streaming, high volume, replay	Cloud-native pub/sub with light ops

Reach for RabbitMQ when routing logic is the hard part: work queues, per-message fan-out, priority handling, retries with dead-lettering. Reach for Kafka when you need a durable, replayable event log at high volume. RabbitMQ's Streams close some of that gap, but Kafka still owns the log-first world.

Clustering and HA

RabbitMQ nodes form a cluster that shares metadata (exchanges, bindings, users). Queue data is a different matter.

- Classic queues live on one node. Lose the node, lose the queue unless mirrored (deprecated).
- Quorum queues replicate their contents across an odd number of nodes (3 or 5) using Raft. This is the supported path to HA.
- Put a load balancer in front, but connect clients to the cluster, not a single node.

For a fault-tolerant setup: quorum queues, durable declarations, persistent messages, and publisher confirms. Any one of those missing and you have a gap.

Best practices

- Reuse connections, open a channel per thread. One TCP connection is expensive. Channels are cheap and multiplexed over it. Never share a channel across threads.
- Always set prefetch. Unbounded prefetch lets one consumer starve the rest and blow up memory.
- Ack manually for anything that matters. Auto-ack trades your durability for a little speed.
- Use both durable queues and persistent messages. Half the contract is no contract.
- Wire a dead-letter exchange from day one. It is the only clean way to handle poison messages.
- Set queue length limits. An unbounded queue turns a slow consumer into an out-of-memory broker.
- Use quorum queues for HA. Classic mirrored queues are gone in 4.0.
- Make consumers idempotent. At-least-once delivery means duplicates will happen.
- Bound your topics. A binding per user creates thousands of bindings and slows routing.

Common gotchas

- Durability needs two flags. Durable queue plus persistent message. Miss either and a restart eats the data.
- Auto-ack silently drops messages. A crash after delivery but before processing loses the message with no trace.
- No prefetch means no fairness. The first consumer grabs the whole backlog while others idle.
- Channels are not thread-safe. Sharing one across threads corrupts the protocol framing. One channel per thread.
- Unacked messages pile up. A consumer that never acks holds messages as “unacked” forever, and prefetch eventually blocks it entirely.
- Requeue loops. `basicNack` with `requeue=true` on a message that always fails creates an infinite loop. Use a DLX with `requeue=false`.
- No native exactly-once. Confirms plus acks give at-least-once. Dedupe on the consumer if you need effectively-once.
- Mirrored queues are deprecated. Do not build new systems on them. Use quorum queues.
- The default exchange hides routing. Publishing straight to a queue name works, but the routing is invisible and unmaintainable at scale.
- Unroutable messages vanish. Without the `mandatory` flag and a return listener, a message with no matching binding is dropped without error.

Quick reference

Concept	Key fact
Exchange	Routes messages, never stores them
`direct`	Routing key equals binding key
`topic`	Wildcard match, `*` one word, `#` zero or more
`fanout`	Broadcast to all bound queues
`headers`	Route on headers, not routing key
Default exchange	Nameless, queue auto-bound by name
Binding	Rule connecting an exchange to a queue
Queue	Ordered buffer, holds until acked
Quorum queue	Raft-replicated, the HA default
Stream	Replayable append-only log
Durability	Durable queue plus persistent message
Prefetch	`basicQos`, cap on unacked messages
Ack	`basicAck`, `basicNack`, `basicReject`
DLX	Dead-letter target for rejected or expired messages
Confirms	Broker acks that a publish was stored
Delivery	At-least-once, dedupe for effectively-once
Connection vs channel	One connection, one channel per thread
Java client	`com.rabbitmq:amqp-client`